Identifying future bugs from past testing

Specifications

octo

Devin Fellhauer, Jonas Mechnig, Franziska Killius,
Felix Kellenbenz, Sefa Burak Yildiz

supervised by
Miklós Borsi, Dennis Kobert

Contents

1 Introduction **2**

2 Target Determination **3**

3 Product Application **5**

4 Product Environment **8**

5 Development Environment **9**

6 Product Overview **10**

7 Product Functions **11**

8 Non-functional Requirements **32**

9 Product Data **35**

10 User Interface **38**

11 Test Scenarios **42**

12 Glossary **44**

1 Introduction

Testing is a very important process in software projects. The aim of this project is to provide software developers with the possibility to find bugs more easily, thanks to the designed framework.

The framework is able to run tests of the project repeatedly. It can also print the results and then create a dataset based on those results, which can be used to train a ML model. On the other hand, the framework determines the impact of commits on the success or failure of tests by considering the all specific changes which made. Furthermore, it provides that tests can also be run on any version of the given project.

In conclusion, we hope to be able to help all developers find their bugs, especially at the projects those that are increasing in size and feature count and involve large teams.

2 Target Determination

The goal is to create a framework with the ability to repeatedly execute a project's tests and produce the results. It is also able to create a dataset, based on these results. On the one hand, this data is used to train a ML model. On the other hand, it enables developers to quickly locate bugs and gain a comprehensive overview of the project's history.

Must Criteria

/M10/ Run a programs test functions repeatedly to analyzed the project in different revisions.
/M20/ Build and export a data set with test results and metadata.
/M30/ Standardized format for exported files and the output to stdout.
/M40/ Identify lines between versions that are relevant to the test outcome.
/M50/ Accumulate history of tests.
/M60/ Support projects that use JUnit as a testing framework.
/M70/ Ignore specific files or tests from being analyzed by the framework.
/M80/ Do not modify the underlying projects source code.
/M90/ Add filters to view metadata selectively based on the specified filter.

Wish Criteria

/W10/ Containerize the tested code.

/W20/ Make the application usable for different programming languages.

/W30/ Ensure multi-platform support.

/W40/ Usable with multiple common testing frameworks.

/W50/ Support displaying a quick review, a table of tests and commits.

/W60/ Scan frequently modified code sections for potential weakness.

/W70/ Include code coverage analysis into the test analytics.

/W80/ Provide output in multiple formats.

Demacration Criteria

/D10/ Support simultaneous testing of multiple projects.

/D20/ Support testing frameworks that are uncommon.

/D30/ Run outside the git directory.

/D40/ Store the data on a server.

/D50/ Train the machine learning model.

3 Product Application

Scope of Application

The core functionality of our product revolves around empowering developers to conduct thorough tests on their projects. Once the tests are executed, our framework meticulously analyzes the results in relation to the current version of the project. Users have the flexibility to choose how the framework processes the results, such as retrieving the test history, running all tests in a specific commit, or comparing test outcomes across different commits.
Post-test execution, the program presents the analyzed results in a standardized format. What sets our product apart is its capability to generate data that can be harnessed for training a Machine Learning Model. This model is specifically designed to detect and identify potential future bugs.
Moreover, this data provides developers with a valuable tool for effortlessly pinpointing bugs and gaining a swift overview of the project's historical performance. The seamless integration of testing, analysis, and machine learning empowers developers to enhance the reliability and stability of their projects.

Target Group

The main target group for our project includes software developers, quality assurance teams and development managers whose projects use git repositories. The tool is aimed at teams who are interested in improving the development process by gaining insight into the impact of code changes on test results. In particular, the tool is designed to support data-oriented development teams. The project aims to find errors more easily and quickly in large software projects as the project size and functionality increases and new people join the project. For example, large software companies, software engineers, startups, academics and independent researchers are among the groups that make up our target audience. Furthermore, the data provided by the tool can be used to train a machine learning model.

Operating Conditions

Framework Operating Environment:

The framework is designed to operate effectively in diverse IT environments. It is designed to work seamlessly in localised development environments as well as in potential containerisation scenarios. We presume that the required runtime environment and libraries for the tested software are already available.

Framework Operating Time:

To ensure reliable data acquisition over multiple test cycles, the framework is designed for continuous operation. The daily operation time vary depending on the testing process requirements. However, the system handles repeated test runs without experiencing significant performance degradation.

Resource Requirements:

The framework uses appropriate resources such as CPU, memory and storage, to ensure efficient test execution and reliable data processing. The exact resource requirements may vary depending on the size of the tested software and the complexity of the test cases.

Data Management and Privacy:

The application is not allowed to introduce new privacy or security issues by copying data from the project under analysis to other directories or by exporting the data elsewhere.

Flexibility and Adaptability:

The framework must be flexible enough to adapt to different software projects. This may require alterations in programming languages, tested libraries, or other specific prerequisites. For the generalisation of research results to different programs, the adaptability of the software is significant.

Documentation and Training:

Comprehensive documentation for the framework needs to be accessible to ensure easy handling and maintenance. If needed, training resources must be supplied to help users make the most of the framework.

4 Product Environment

Software Environment

Octo can be executed on a Unix-based operating system by simply installing the octo application and run it in the CLI.

Hardware Environment

Depending on the number of commits of the program that should be tested there should be enough space to store test results. Other than that, there are no hardware requirements.

5 Development Environment

Software Environment

Operating System

- Microsoft Windows
- MacOS 13
- Linux Ubuntu

Development

- Eclipse
- IntelliJ
- Maven

Version Management

- Git
- GitHub

Other Software

- Draw.io
- UMLetino

Package Manager

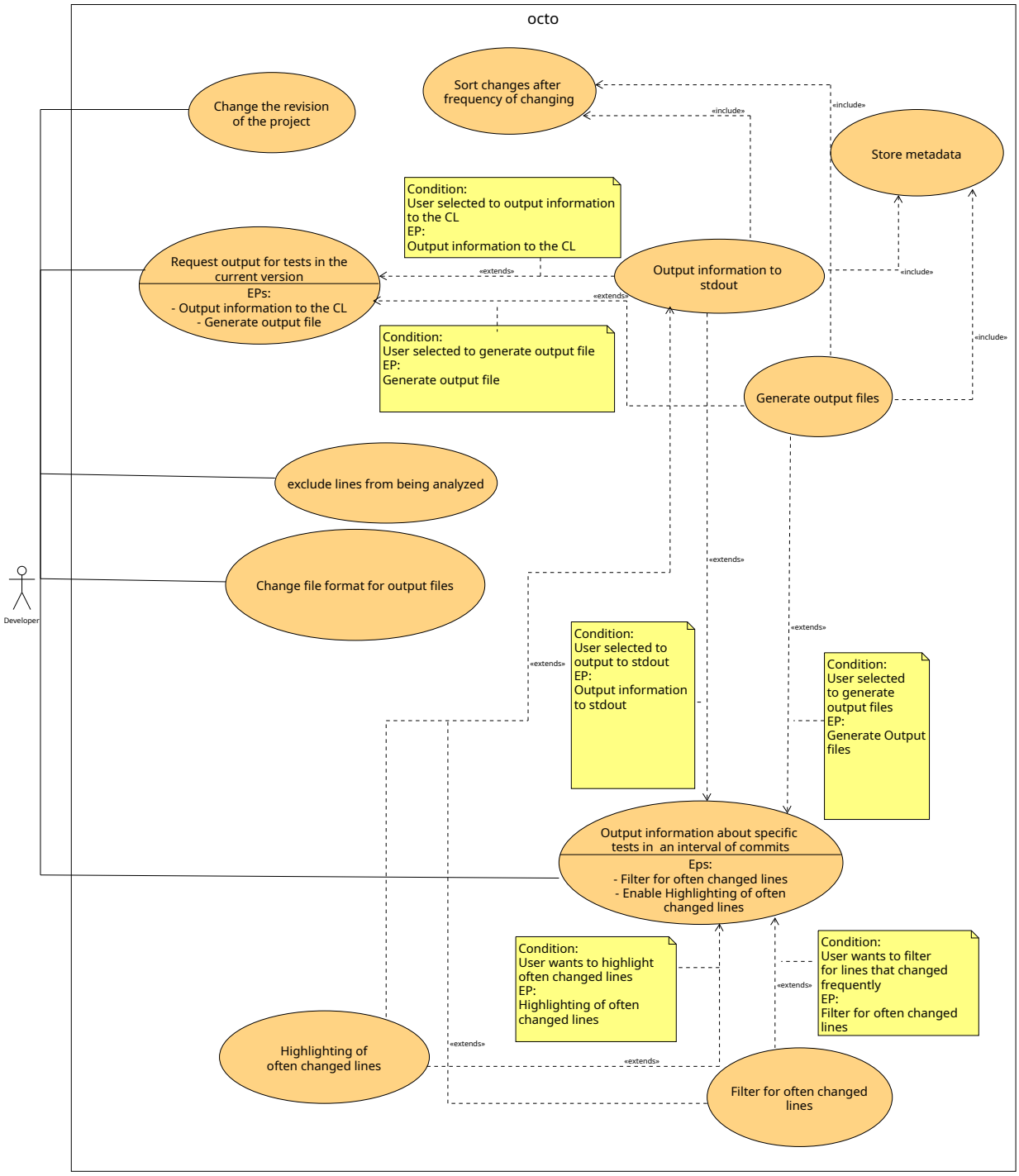
- Maven

Hardware Environment

No specific or specialized hardware was used.

6 Product Overview

Use Cases



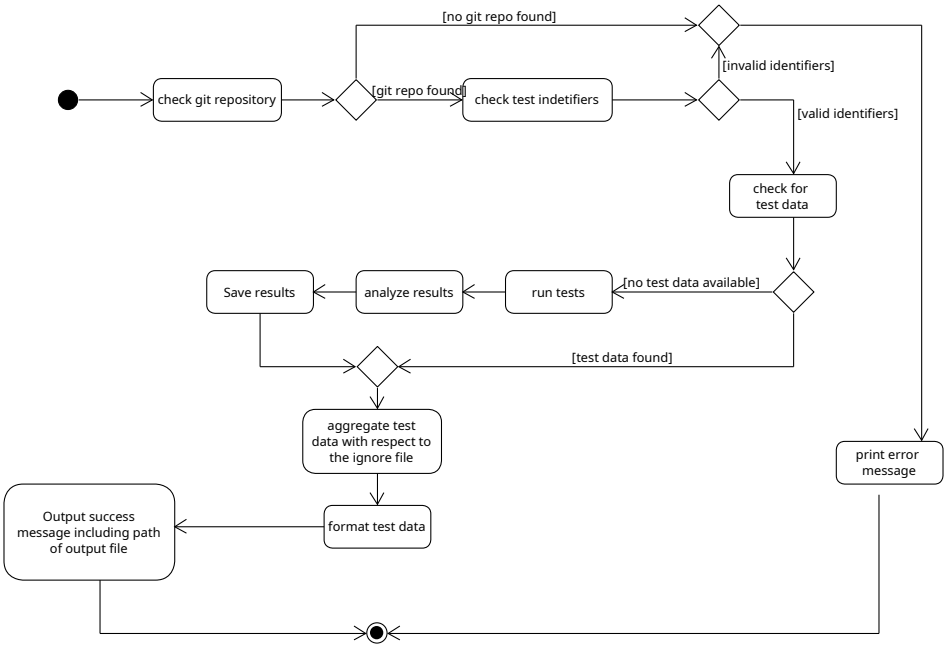
7 Product Functions

In this section, the individual functions of the project are further defined.

/F10/	Output tests from a single version to a file
/F20/	Output tests from a single version to the CLI
/F30/	Change the revision of the project to be tested
/F40/	Analyze an interval of commits
/F50/	Filter often changed lines
/F60/	Prioritize tests with alternating results
/F70/	Modify the .octoignore file
/F80/	Change the default format of the output file
/F90/	Store metadata
/F100/	Highlight often changed lines
/F110/	Store user settings in a config file

/F10/ Output tests from a single version to a file

1. **Title:** Output information about the specified tests.
2. **Short description:** This allows the user to input a command to request an output file for specified tests.
3. **Actors:** User
4. **Preconditions:** A git repository is set or the user is currently in a git repository.
5. **Triggering event:** User inputs the command, specifies the tests and the version, then presses the enter key.
6. **Description of execution:** The application will check if the tests have already been run and information about those tests is saved, if so it will use the database entry and generate an output file. If there is no information saved for those tests the application will run the tests and analyze the results. Then it saves the results in a database, generates an output file and prints the location of the file.
7. **Postcondition success:** The output was put in a dedicated directory.
8. **Postcondition failure:** The framework prints an error message via stderr.
9. **Annotation:** -



10. **Tests:**

    (a) Specify non-existent repository → ERROR

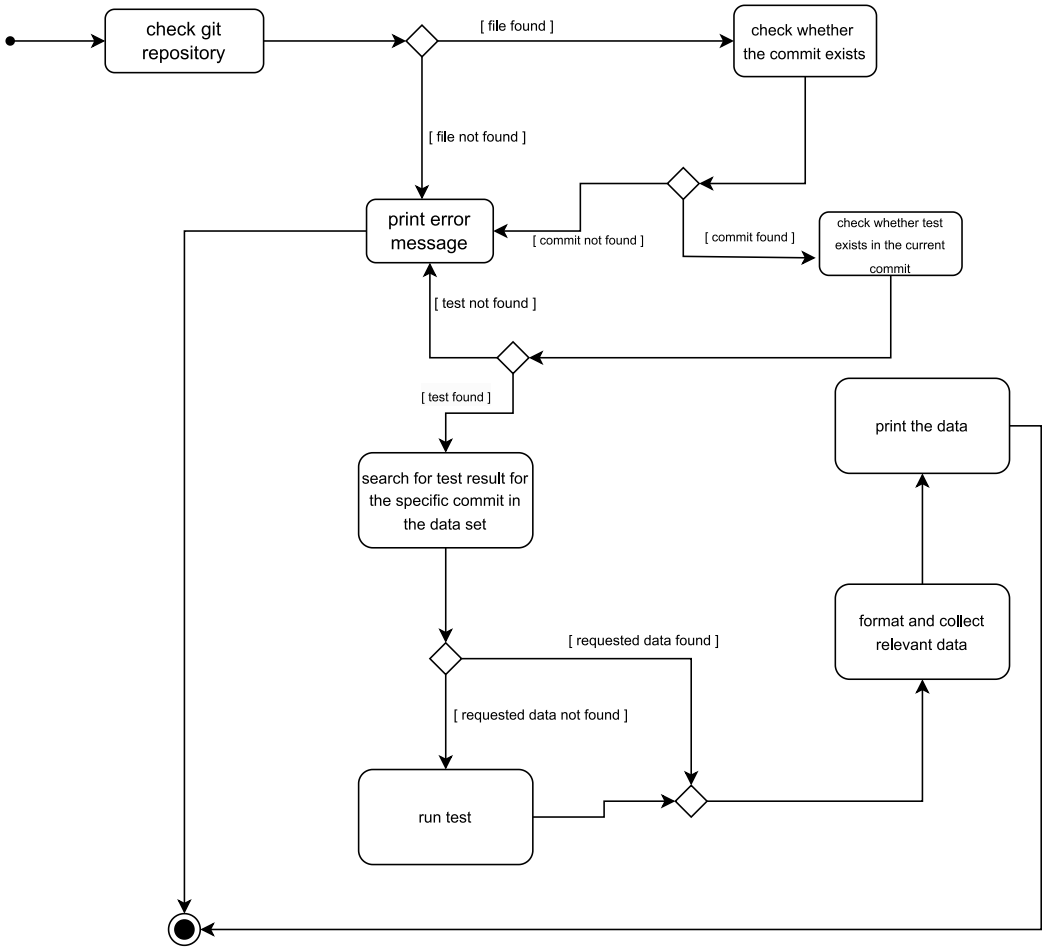
    (b) Specify invalid identifiers → ERROR

    (c) Request tests for which data already exists → format and output test data

    (d) Request tests for which data does not exist → run tests, store data, format and output test data

/F20/ Output tests from a single version to the CLI

1. **Title:** Output metadata to the CLI.
2. **Short description:** Print formatted output on the CLI with information about the specified tests in the specified version.
3. **Actors:** User
4. **Preconditions**: The user is currently in a git repository. Moreover, the user has specified the tests and a valid commit.
5. **Triggering event:** The user enters the corresponding command and the tests, then presses the enter key.
6. **Description of execution:** The framework will search the test result for the commit in the database or run the missing tests. The collected information, including the test result and the possible error lines, is then collected and formatted. Finally, the result is displayed via stdout.
7. **Postcondition success:** The relevant information is displayed via stdout.
8. **Postcondition failure:** The framework prints an error message via stderr, if the test can not be located or no git repository has been found.
9. **Annotation:** -



10. **Tests:**

    (a) Specify non existing file → ERROR

    (b) Specify non existing commit → ERROR

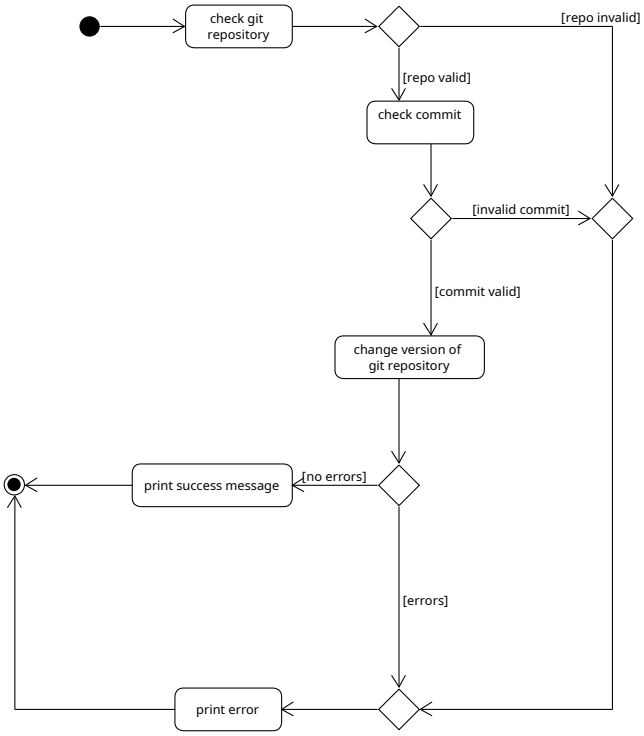
    (c) Specify non existing test → ERROR

    (d) Request tests for which data already exists → format and output test data

    (e) Request tests for which data does not exist → run tests, store data, format and output test data

/F30/ Change revision

1. **Title:** Change the revision of the project.
2. **Short description:** This command allows the user to change the version of the git repository to the commit that was specified.
3. **Actors:** User
4. **Preconditions:** The current working directory is a git repository. Additionally, the user must provide a valid commit.
5. **Triggering event:** User inputs the command associated with this function, specifies all the needed parameters and presses the enter key.
6. **Description of execution:** First, the application checks if the user is currently in a git repository then it checks if the specified commit is a valid commit. within the git repository. Afterwards, HEAD is set to the specified commit.
7. **Postcondition success:** The version of the git repository is changed and a message "project changed to version: <commit>" is output to stdout.
8. **Postcondition failure:** The version of the git repository was not changed and an error message is printed.
9. **Annotation:** -



10. **Tests:**

    (a) Specify invalid repository → ERROR

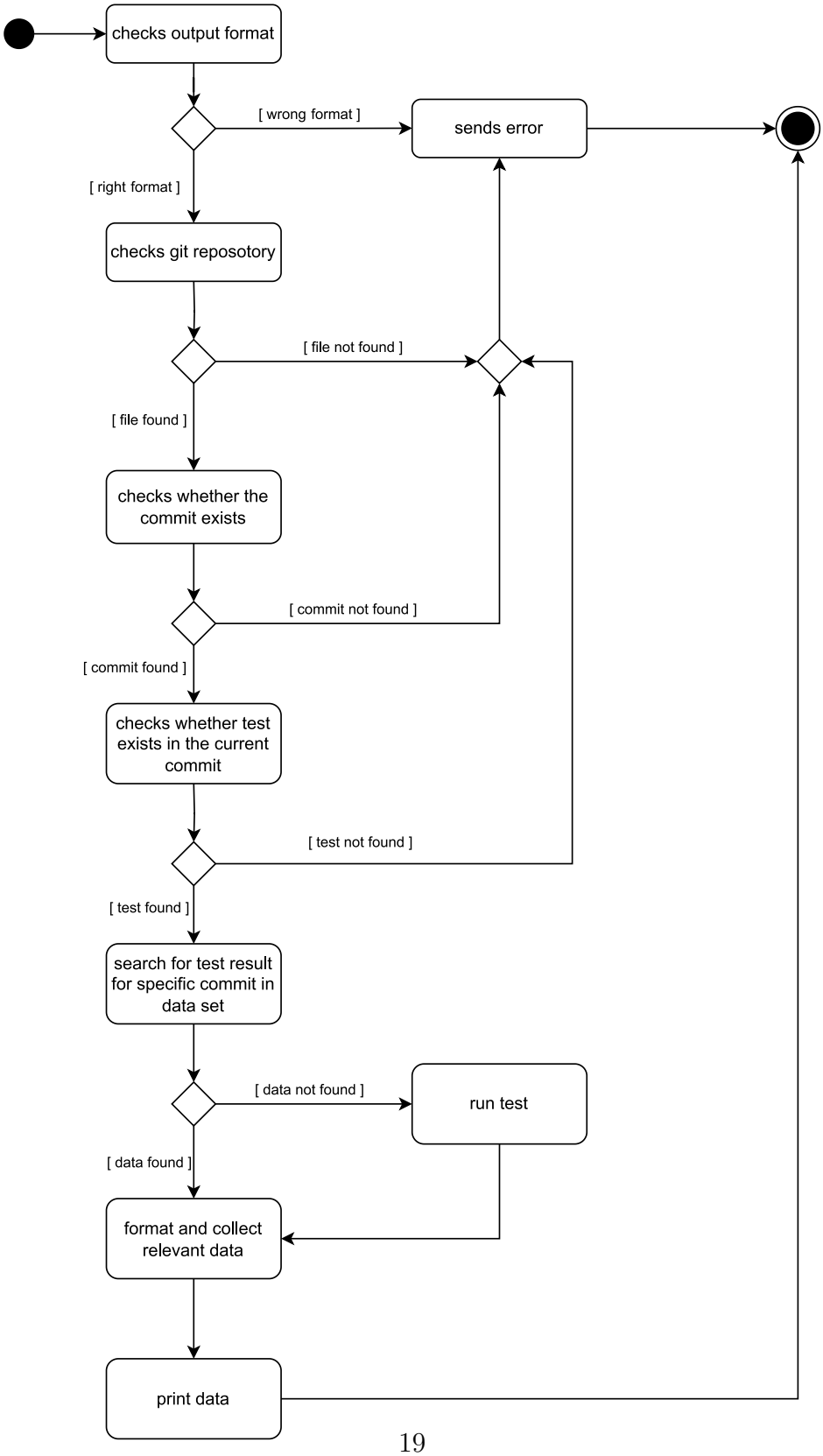
    (b) Select invalid commit → ERROR

    (c) Create errors while trying to change the version of git repository → ERROR

    (d) Successfully change the version of git repository → print success message

/F40/ Analyze an interval of commits

1. **Title:** Print analyzed results of specific tests in an interval of commits.
2. **Short description:** This command lets the user select a set of tests and commits. The results are then printed to the command line or exported as a JSON file.
3. **Actors:** User
4. **Preconditions:** The user indicates the test for which they want to obtain results and specifies the commits from the repository.
5. **Triggering event:** The user passes a command to the command line, which contains a possible set of tests as well as a range of commits.
6. **Description of execution:** First, the application accepts the input and checks whether it has the correct input format. Then, the git repository is checked. After that, the specified commits and the specified tests are searched for. If the data for the tests do not exist, the tests are executed. The data is then collected, formatted and printed.
7. **Postcondition success:** The data requested by the user is output via stdout.
8. **Postcondition failure:** An error is displayed and the user is prompted to enter a new command.
9. **Annotation:** -


checks output format
[wrong format]
sends error
[right format]
checks git reposotory
[file not found]
[file found]
checks whether the commit exists
[commit not found]
[commit found]
checks whether test exists in the current commit
[test not found]
[test found]
search for test result for specific commit in data set
[data not found]
run test
[data found]
format and collect relevant data
print data

10. **Tests:**

    (a) Specify wrong input format → ERROR

    (b) Specify non existing repository → ERROR

    (c) Specify non existing commit → ERROR

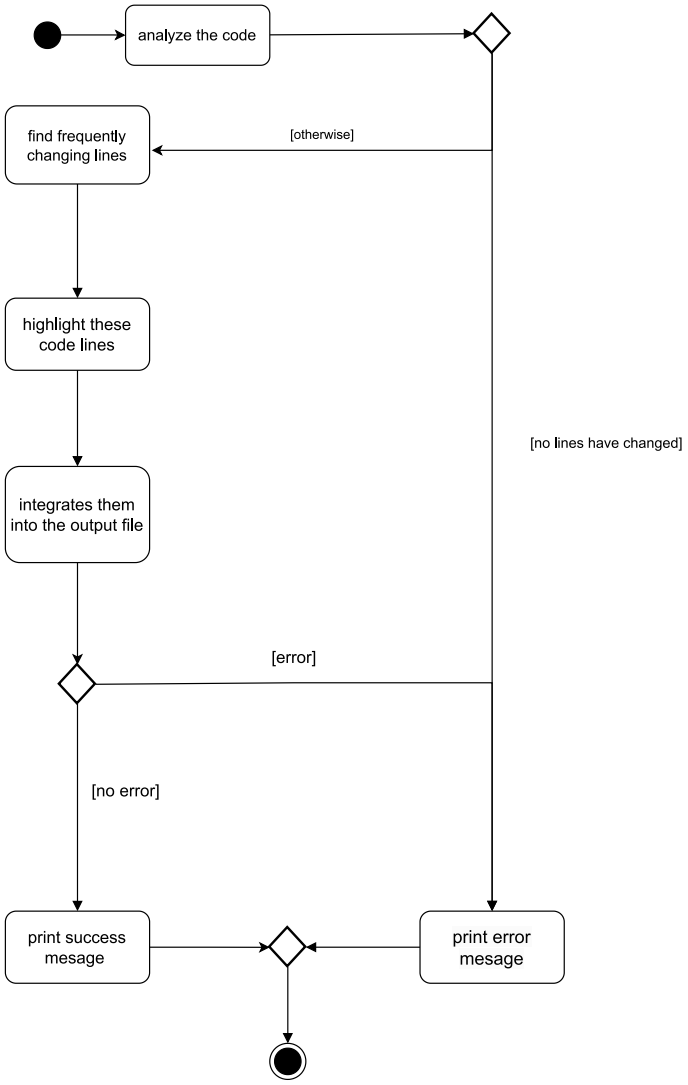
    (d) Request non existing test → ERROR

    (e) Request tests whose data already exists → format and output test data

    (f) Request tests whose data does not exist → run tests, store data, format and output test data

/F50/ Filter frequently changed code

1. **Title:** Filter lines of code that change frequently.
2. **Short description:** This command allows the user to filter for lines that changed frequently.
3. **Actors:** User
4. **Preconditions:** The user is in a git repository.
5. **Triggering event:** User inputs the command and all the needed information related to this function.
6. **Description of execution:** First, the application finds frequently changing lines of code based on the given parameters from the user. Second, it prints these lines of code in a standard format.
7. **Postcondition success:** Lines of Code that changed more frequently are printed to stdout.
8. **Postcondition failure:** A error message reporting what went wrong is printed to stderr.
9. **Annotation:** -



10. **Tests:**

    (a) Specify non existing part of code → ERROR

    (b) Receive an invalid piece of code → ERROR

    (c) Receive a valid piece of code → print success message

/F60/ Emphasize often changing tests

1. **Title:** Pinpoint specific tests where the output results tend to change frequently.
2. **Short description:** This function allows the user to identify certain test cases whose test results often alternate between true and false.
3. **Actors:** User
4. **Preconditions:** The user is in a git repository.
5. **Triggering event:** A command requesting output to stdout or export metadata files is executed.
6. **Description of execution:** Our program will sort the tests based on the number of times the results of the tests has been changed. This is the default order when printing to stdout or exporting metadata files.
7. **Postcondition success:** Every request to export files or print to stdout has the tests sorted accordingly.
8. **Postcondition failure:** -
9. **Annotation:** -

10. **Tests:**

    (a) Specify invalid repository → ERROR

    (b) Specify non existing tests → ERROR

    (c) Request tests whose data already exists as external file → format and output test data as an external file

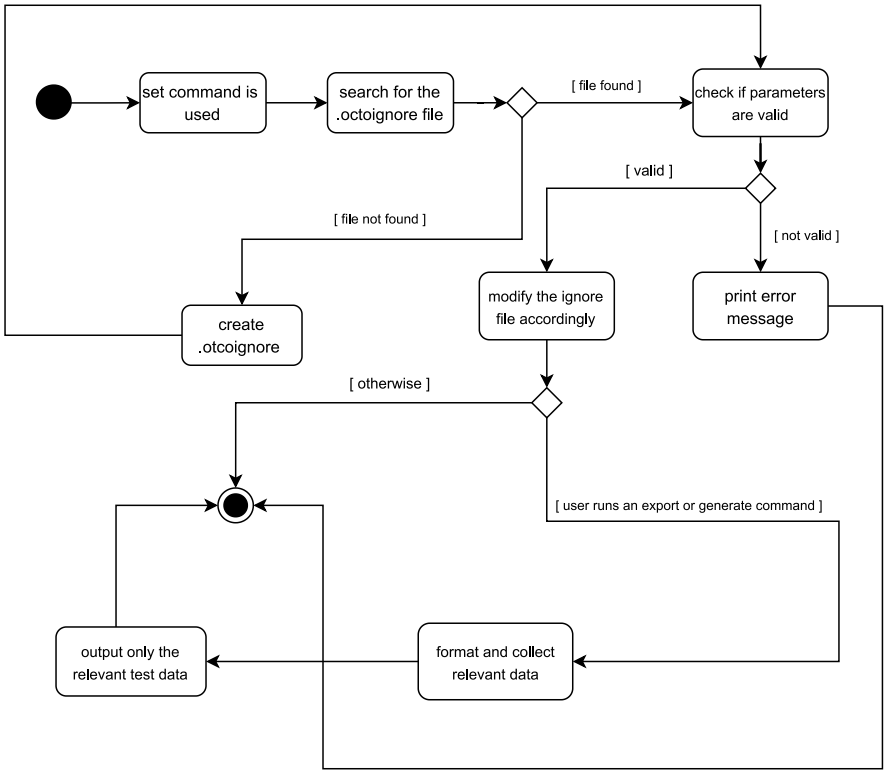
    (d) Request tests as an external file whose data does not exist → run tests, stores data, format and output test data as an external file

    (e) Request tests as command line output whose data already exists → format and output test data via CLI

    (f) Request tests as command line output whose whose data does not exist → run tests, stores data, format and output test data via CLI.

/F70/ Modify the .octoignore file

1. **Title:** Skipping analysis for specified files.
2. **Short description:** The command enables the user to exclude specific project files from analysis. By adding these files to the ignore list, the framework will automatically exclude any corresponding tests from the analysis.
3. **Actors:** User
4. **Preconditions:** The user is in the git repository of the project.
5. **Triggering event:** The user enters a filename into the ignore file and then initiates a command that runs the framework.
6. **Description of execution:** Whenever the set command is invoked, it checks for the existence of the ignore file. If the file does not exist, a new one is created. The file is then modified according to the parameters specified in the set command. If the parameters specified are invalid, an error message is displayed. The ignore file is used to determine whether one or more tests are part of a file listed in the ignore file. In such cases, the identified test(s) will be excluded from the relevant output and ignored by the framework.
7. **Postcondition success:** Only the relevant test results for the user are displayed.
8. **Postcondition failure:** On the stderr a message is displayed indicating that the parameters are not valid.
9. **Annotation:** -



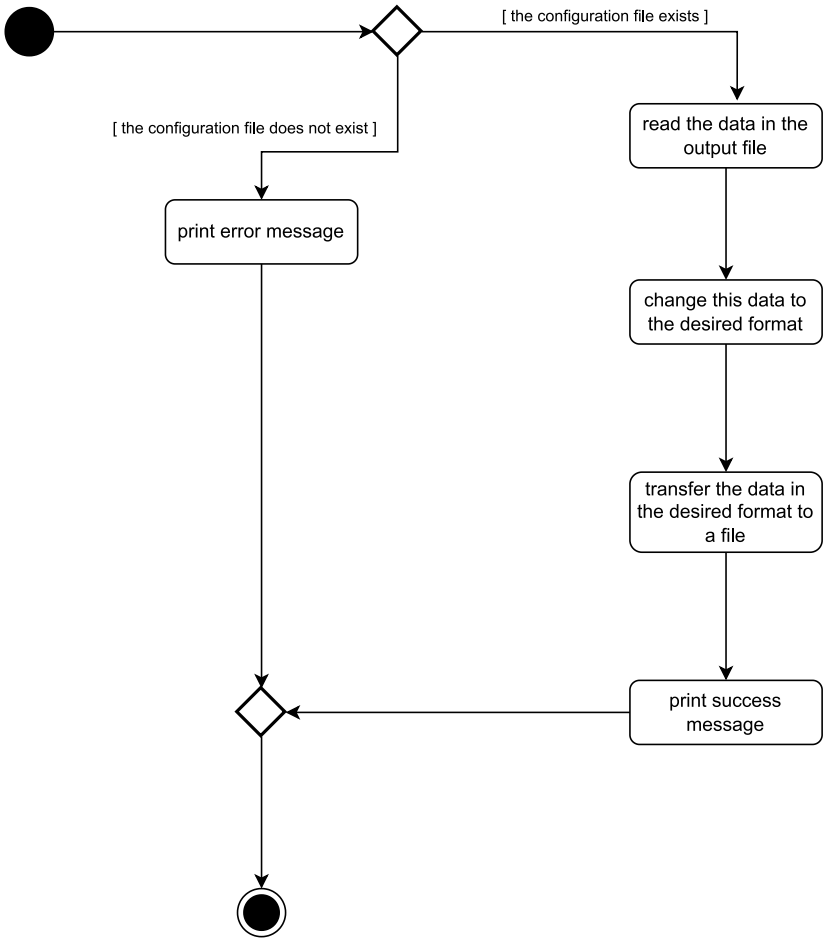
10. **Tests:**

    (a) User requests non existing file → ERROR

    (b) User requests existing file → user runs a command, and receives relevant test data

/F80/ Change default format of the output file

1. **Title:** Change the default format of the output file.
2. **Short description:** The command allows the user to change the default output format of the files that contain data about the test. The framework supports XML and JSON.
3. **Actors:** User
4. **Preconditions:** None
5. **Triggering event:** User enters the command and the desired file format.
6. **Description of execution:** First, we check that the parameter representing the output format is valid, and then we set the default output format to the specified format.
7. **Postcondition success:** The message "The format of the file has been successfully changed." will be printed to stdout. All future exports or outputs to stdout will be in the in the specified output format.
8. **Postcondition failure:** An error message is displayed if the output file can not be changed to the desired format.
9. **Annotation:** -



10. **Tests:**

    (a) User chooses a non existing output format → ERROR

    (b) User chooses an existing output format → returns data set in a new output format

/F90/ Store metadata

1. **Title:** Store metadata.
2. **Short description:** Store metadata on run tests in an SQLite database.
3. **Actors:** None
4. **Preconditions:** Tests have been run.
5. **Triggering event:** The user has requested the export or printing of metadata about tests.
6. **Description of execution:** If the user has requested the export or output to stdout our framework will store data, if none is stored already on those tests the user requested output (to stdout or export) for.
7. **Postcondition success:** Future exports and outputs to stdout will not run all tests again.
8. **Postcondition failure:** The program fails silently and future requests for output to stdout and exports will run those tests again on which no information could be stored.
9. **Annotation:** -
10. **Tests:** -

/F100/ Highlight often changed lines

1. **Title:** Highlight lines of code.
2. **Short description:** This gives the user the capability to enable the colorful highlighting of often changed lines of code the output to stdout.
3. **Actors:** User
4. **Preconditions:** The user is currently in a git repository.
5. **Triggering event:** The user runs the associated command and presses "Enter".
6. **Description of execution:** The framework utilises picocli to distinguish lines by colour, based on the number of alterations. For example, if a line undergoes frequent changes, the content is highlighted in organge in the "Changes" section of the output.
7. **Postcondition success:** The message to stdout includes the highlighting of lines of code that changed often.
8. **Postcondition failure:** There are no colors in the message that gets printed.
9. **Annotation:** -
10. **Tests:** -

/F110/ Store user settings in a config file

1. **Title:** Store the settings for octo in a config file.
2. **Short description:** The framework will store any settings in a config file, every other function will consider the config settings when executing a function.
3. **Actors:** User
4. **Preconditions:** None
5. **Triggering event:** The user requests octo to run one of its features.
6. **Description of Execution:** Everytime octo executes a function described by the functional requirements above it checks if a config file exists. If no config file exisits one is created in the installation directory of octo and initialized with the default values.
7. **Postcondition success:** octo behaves as specified in the config file.
8. **Postcondition failure:** Fails silently.
9. **Annotation:** -
10. **Tests:** -

8 Non-functional Requirements

/NF10/

1. **Title:** Speed
2. **Short description:** The system should be interactive and users should not wait seconds for output or results.
3. **Reason:** The interaction with our system should be smooth and pleasant for the user.

/NF20/

1. **Title:** Protect the given project and data
2. **Short description:** The program will not change any assets of the project. The project is only read and not overwritten.
3. **Reason:** Our system is designed to analyze the test behavior of a project and collect the data. The given project should not be changed in any way. Otherwise it could damage the project and lead to data loss

/NF30/

1. **Title:** Minimum performance
2. **Short description:** The system should be able to handle projects with up to 1000 commits.
3. **Reason:** For the user to be able to work well with our system, it should be able to process at least 1000 commits. However, the user is given the option of sending more than 1000 commits, but receives a warning that this could lead to the program crashing.

/NF40/

1. **Title:** Easy usage
2. **Short description:** The CLI should be easy and intuitive to use.
3. **Reason:** The CLI should be easy to use for the user so that he can work efficiently. It is important to bear in mind that the user will be a developer themselves and will therefore have useful prior knowledge in certain areas.

/NF50/

1. **Title:** Easily extendable
2. **Short description:** Octo should be easy to extend.
3. **Reason:** As octo will largely be used by developers, it stands to reason that they will be able to add their own commands to the program. To make this as easy as possible for the user, the code should be structured in such a way that custom commands are easy to implement.

/NF60/

1. **Title:** Language
2. **Short description:** All interactions (output etc.) are in English.
3. **Reason:** English is the universal language, making it applicable to numerous projects and enhancing user interaction.

/NF70/

1. **Title:** Low maintenance needs
2. **Short description:** The program should be easily repairable and maintainable if necessary.
3. **Reason:** It is important for program availability, lifetime and customer satisfaction that the program can be quickly repaired in case of problems or errors.

9 Product Data

The framework will store information about the tests that have already been run to avoid running every test every time.

```
{
  "CommitId": "123abcde",
  "previousCommitId": "789def",
  "tests": [
    {
      "id": 1,
      "contents": "assert( x=5);",
      "passes": false
    },
    {
      "id": 3,
      "contents": "assert( y!=2);",
      "passes": true
    }
  ],
}
```

Listing 1: JSON output by using the command generate and not the flag –verbose

```
Testing Summary:
      Total Tests: 2
      Passed: 1
      Failed: 1
      Test Coverage: 42%

Code Changes:
      Files Changed: 1
      Total Lines Changed: 4
      Insertions: 8
      Deletions: 0

Contributors:
      Author: Admiral Ackbar

```

Listing 2: Example for a possible stdout via the analyse command without the flag –verbose

```
{
  "CommitId": "123abcde",
  "previousCommitId": "789def",
  "changes": [
    {
      "filename": "abc.java",
      "linesChanged": [12, 13, 14, 15],
      "textOfChanges": [
        "for (int i=0; i<5; i++) {",
        "  System.out.println(\"Iteration: \" + i);",
        "  if (i % 2 == 0) {",
        "    System.out.println(\"Even number\");",
        "  } else {",
        "    System.out.println(\"Odd number\");",
        "  }",
        "}"
      ]
    }
  ],
  "tests": [
    {
      "id": 1,
      "contents": "assert( x=5);",
      "passes": false
    },
    {
      "id": 3,
      "contents": "assert( y!=2);",
      "passes": true
    }
  ],
  "additionalInfo": {
    "author": "Admiral Ackbar",
    "timestamp": "2021-12-25T12:28:56",
    "commitMessage": "It's a trap",
    "testCoverage": "42%",
  }
```

Listing 3: Format for JSON output files by using the command generate and the flag –verbose

```
    Testing Summary:
          Total Tests: 2
          Passed: 1
          Failed: 1
          Test Coverage: 42%

    Code Changes:
          Files Changed: 1
          Total Lines Changed: 4
          Insertions: 8
          Deletions: 0

    Contributors:
          Author: Admiral Ackbar

    Commit ID: 123abcde
    Previous Commit ID: 789def

    Changes:
          Filename: abc.java
          Lines Changed: [12, 13, 14, 15]
          Text of Changes:
            for (int i=0; i<5; i++) {
              System.out.println("Iteration: " + i);
              if (i % 2 == 0) {
                System.out.println("Even number");
              } else {
                System.out.println("Odd number");
              }
            }

    Tests:
          Test ID: 1
          Contents: assert( x=5);
          Passes: false

          Test ID: 3
          Contents: assert( y!=2);
          Passes: true

    Additional Information:
          Timestamp: 2021-12-25T12:28:56
          Commit message: It's a trap


```

Listing 4: Example for a possible stdout via the analyse command and the flag –verbose

10 User Interface

Users of octo will interact with it via the CLI. In the following possible user interactions are described.

The octo Help Command

```
[user]@[personal-machine]:$ octo --help

Run tests of a project

    octo analyse [OPTIONS] <tests>

    octo generate [OPTIONS] <start> <end> <tests>

    octo project-scan [OPTIONS]

    octo project-scan [--file] [--out] [--verbose] [--help]

Information about the current setup of octo

    octo set [OPTIONS]

    octo get
```

Listing 1: man page for the octo application

The octo Generate Command

```
NAME
        generate

SYNOPSIS
        octo generate [OPTIONS] <start> <end> <tests>

DESCRIPTION

        Generates metadata files for specified <tests>
        in an interval of commits from <start> to <end>,
        the default of <tests> is all tests in one commit.
        If only one commit is specified then the specified
        tests or the default gets analyzed in the
        specified commit. To analyze an interval of commit,
        you can either choose to specify a start and end
        commit or a end and an offset. If no special format
        was specified the default format is used. See octo
        set [OPTIONS] tosee how to change the default format.

        Mandatory arguments to long options are mandatory
        for short options too.

OPTIONS
        -v, --verbose
            The generated file contains additional
            information such as author, timestamp, commit
            messages and the lines which have changed.

        -f, --format <format>
            Use the specified format as format for the
            files possible paramters are
            'XML' and 'JSON'.

        -h, --help
            Display this help and exit.
```

Listing 2: man page for the generate command of octo

The octo Analyse Command

```
NAME
        analyse

SYNOPSIS
        octo analyse [OPTIONS] [<start>] <end> <tests>

DESCRIPTION

        Output the analyzed tests to stdout for an interval
        of commits or a single commit. The default value for
        <tests> is all tests in a commit. If only one commit
        is specified then the specified tests or the default
        get analyzed in the specified commit. To analyze an
        interval of commits, you can either choose to
        specify a start and end commit or an end commit and
        an offset. The metadata is the output to the CLI
        respecting the output options.

        Mandatory arguments to long options are mandatory
        for short options too.

OPTIONS

        -v, --verbose
            The output to stdout contains additional
            information such as author, timestamp,
            commit messages and the lines which have
            changed.

        -hl, --highlight
            Provides highlighting of frequently amended
            lines in the "Changes" section of the output.

        -fq, --frequency <frequency>
            With the frequency option one can specify how
            often a line has to change to be part of the
            output. The default value is 0.

        -h, --help
            Display this help and exit.
```

Listing 3: man page for the analyse command of octo

The octo config Command

```
NAME
        config

SYNOPSIS

        octo config

DESCRIPTION

        Prints information about the current octo setup
        to the CLI.

        Mandatory arguments to long options are
        mandatory for short options too.
```

Listing 4: man page for the config command of octo

11 Test Scenarios

/TS10/ **Get a quick overview of the current project.**

1. The user navigates in the CLI to the directory where the project is stored.
2. They run the octo analyse command
3. The framework will analyse the code and, depending on the flag, add more detail to the output, which is displayed via stdout.
4. The user sees the summary and gets an overview of the project bugs.

/TS20/ **Apply changes to the default values of octo**

1. The user runs the octo set command
2. They pass the parameters of the defaults that he wants to overwrite.
3. Octo overrides the default settings and the new presets are applied when the user executes the next commands.

/TS30/ **Generate a JSON file.**

1. The user runs the octo generate command with the appropriate flags within the directory of the project.
2. The command collects the relevant information.
3. This information is then formatted and if specified a filter is applied.
4. A JSON file is generated in the directory which contains the data.

/TS40/ **Add files to .octoignore.**

1. The user executes the octo set command and uses the -i flag to specify the files he wants to exclude from the data provided by octo.
2. Before data is output, octo checks the .octoignore file and omits any data from these files in the output.

12 Glossary

CLI Command Line Interface. 8, 11, 14, 24, 33, 38, 42

Eclipse Eclipse is an open-source integrated development environment (IDE) used for Java development. 9

git Git is a version control system used for collaborative software development, allowing easy tracking and management of revisions and changes. 4, 6, 9, 12, 14, 16–18, 21, 23, 25, 30

GitHub GitHub is an online platform for git-based version control that simplifies collaborative software development. It features code hosting coupled with tools for pull requests and issue tracking. 9

IntelliJ IntelliJ IDEA is an integrated development environment (IDE) designed for Java development. 9

JSON JSON (JavaScript Object Notation) is a format for exchanging data that is lightweight and easy to read. It is commonly used to represent structured data as key-value pairs. 18, 27, 35, 36, 43

JUnit JUnit is an open-source Java framework that simplifies the creation and execution of automated tests. 3

Maven Maven is a build automation and project management tool that simplifies the software development process by managing project dependencies, building, and project lifecycle. 9

ML Machine learning. 2, 3

octo Name of the application. 8, 31, 33, 38–43

stderr The "standard error" stream is used for displaying error messages and diagnostic information separate from the standard output. 12, 14, 21, 25

stdout The "standard output" refers to the default stream where a program writes its output in a command-line environment. 3, 14, 16, 18, 21, 23, 27, 29, 30, 35, 37, 42

XML XML (eXtensible Markup Language) is a markup language used to encode and structure data in a text format, using customisable tags to define elements and their hierarchical relationships. 27